Project Lab Logbook: Open Authorization Protocol

02244 Logic for Security

Fahmida Khan (s250116) Zeinab Ali Mithra Arul

March 16, 2026

Abstract

This logbook documents the iterative development of an Open Authorization protocol. The protocol evolved from a simplified Dolev–Yao model to a more complete design including an Identity Provider, challenge-response authentication, and protection of a weak password using cryptographic hashing and session challenges.

1 Introduction

The goal of this project is to design a protocol where a user A allows a service p to access selected files stored on server B. Authentication is handled through an Identity Provider I, and the protocol must ensure that only authorized data is released while protecting the user's weak password.

2 Week 2: 09-02-26 (Author: Zeinab Ali)

Initial Protocol Idea

User A has data stored on server B. A wants service p to retrieve selected files from B. In this initial phase, we simplified the environment by assuming A possessed a private key to sign grants directly.

```
Protocol: OpenAuth_W2_Simple

Actions:
  A -> p: {A,B,p,F,T}inv(pk(A))
  p -> B: {A,B,p,F,T}inv(pk(A)), F
  B -> p: {Photos}pk(p)
```

This version helped establish the basic three-party message flow but was unrealistic as A should not have a private key in the final scenario.

3 Week 3: IDP and Lazy Intruder (Author: Fahmida Khan)

Design Changes

We introduced the Identity Provider I. A now shares a weak password $pw(A, I)$ with I. Authentication is performed using a hash: $h(pw(A, I), NA)$.

Lazy Intruder Analysis

We verified that role p is executable even when played by the intruder (i).

Trace Analysis: 1. i receives $\{A, p, F, T, NA\}_{inv(pk(I))}$ from A. 2. i can compose the outgoing message to B because it simply forwards the signed token it received. This confirms that the service role can be fully automated or impersonated by an attacker once a valid token is released.

4 Week 4: Type-Flaw Resistance (Author: Fahmida Khan)

To prevent type-flaw attacks, we introduced explicit function symbols to differentiate message types, ensuring that a "Grant" cannot be confused with "Photo Data." We also implemented a `GetPublicKey` auxiliary protocol to model how A retrieves $pk(B)$ and $pk(p)$ from the trusted IdP.

5 Week 6: Challenge Response and Privacy (Author: Mithra Arul)

Initial Week 6 Version and Attack Found

We introduced the pseudonymous channel (`*->*`) and a fresh challenge NI. However, our initial AnB attempt resulted in:

```
SUMMARY: ATTACK_FOUND
GOAL: secrets
```

Attack Logic: The intruder intercepted the request and signed a fake grant using `inv(pk(i))`. Because Agent p was not strictly checking that the signature originated from $pk(I)$, it accepted the fake grant and released the secret `Photos` to the intruder.

6 Final Hardened Protocol

The final version resolves previous attacks by binding all session parameters to the password hash and ensuring all agents strictly verify the IdP's signature. We also used `Function::` type-casting to ensure OFMC executability.

```
Protocol: week6_guessablepassword_hardened

Types:
  Agent A,B,I,p;
  Number F,T,R,NI;
  Function pk,pw,h,fGrant,fData,Grant,Photos;

Knowledge:
  A: A,B,I,p,pk(I),pw(A,I),h,fGrant,fData,Function::Grant;
  B: A,B,I,p,pk(B),pk(I),pk(p),inv(pk(B)),fGrant,fData,Function::Grant,
     ↪ Function::Photos;
  p: A,B,I,p,pk(p),pk(I),inv(pk(p)),fGrant,fData,Function::Grant,
     ↪ Function::Photos;
  I: A,B,I,p,pk(B),pk(p),pk(I),inv(pk(I)),pw(A,I),h,fGrant,fData,
     ↪ Function::Grant;

Actions:
  p -> A: p,B,F,T,R
  A *->* I: A,p,B,F,T,R
  I *->* A: NI
  A *->* I: h(pw(A,I), A,p,B,F,T,R, NI)
  I -> p: {fGrant(A,p,B,F,T,Grant)}inv(pk(I))
```

```
  p -> B: {fGrant(A,p,B,F,T,Grant)}inv(pk(I)),F
  B -> p: {fData(Photos)}pk(p)

Goals:
  pw(A,I) guessable secret between A,I;
  secrecy of Photos;
  p authenticates I on Grant;
  B authenticates I on Grant;
  I authenticates A on NI;
```

7 Reflection

We encountered and resolved several modeling hurdles:

- **Knowledge Gaps:** We initially forgot to give B the public key $pk(p)$, making it impossible for B to encrypt the response.
- **Type-Casting:** We used `Function::` casting for `Grant` and `Photos` to allow their inclusion in initial knowledge without violating OFMC agent-typing rules.
- **Salted Hashes:** Adding NI to the hash successfully protected the guessable password from offline dictionary attacks.

8 Resource Disclosure

- 02244 Logic for Security course lectures and slides.
- OFMC / AnB documentation.
- Gemini AI: Assisted in LaTeX formatting, structural organization, and debugging OFMC executability errors.